Good. Now I have read the full `tech_Des.md` which contains Kishore's exact implementation. Let me give you a precise analysis — what specifically is wrong, why it causes false positives, and exactly what to change.

What Kishore Has Built — Exact Specs

From `tech_Des.md`:

```
Phase A: distance < 120px    candidate
Phase B: trajectory intersection + speed drop ≥ 70% + angle > 40°
Phase C: optical flow > 2.5× + bbox deform + flow std dev > 45°
Tracker: custom SORT-like, IoU 0.30, grace period 10 frames
LSTM:    BiLSTM, 16-frame buffer, fused threshold 0.60
```

Problem 1 — The Most Critical Bug: No Trajectory Stop Check

What Kishore has: Phase B checks if trajectory paths intersected. Full stop.

What is missing: The IITH 2018 paper's core insight — the difference between collision and occlusion is what happens AFTER the intersection, not the intersection itself.

In Coimbatore junction traffic, every pair of vehicles' paths intersect constantly — turning left, turning right, overtaking. Trajectory intersection alone is meaningless. It fires on every normal crossing.

What the IITH paper says:

```
Intersection + both trajectories CONTINUE    occlusion    ignore
Intersection + one trajectory STOPS          collision    flag
```

Fix to add to Phase B:

```
def check_trajectory_stop_after_intersection(va, vb, n_frames=5):
    # After detecting intersection, check what happens next
    # If either vehicle's speed drops to near-zero in the
    # frames following the intersection point    real collision

    recent_speed_a = np.mean(va.speed_history[-n_frames:])
    recent_speed_b = np.mean(vb.speed_history[-n_frames:])

    # Both were moving before intersection
    prev_speed_a = np.mean(va.speed_history[-(n_frames+5):-5])
    prev_speed_b = np.mean(vb.speed_history[-(n_frames+5):-5])

    a_stopped = (prev_speed_a > 3.0) and (recent_speed_a < 2.0)
    b_stopped = (prev_speed_b > 3.0) and (recent_speed_b < 2.0)

    return a_stopped or b_stopped  # one stopped = collision
```

Without this check, Phase B will always produce massive false positives in dense Indian traffic.

Problem 2 — Track Grace Period is Too Short

What Kishore has: Custom tracker keeps vehicles alive for only **10 frames** after they disappear.

Why this breaks everything: At 10 FPS, 10 frames = 1 second. In a typical Coimbatore junction with overlapping vehicles, a bike going behind a bus for 1.5 seconds loses its ID entirely. When it reappears it gets a brand new ID with no history. All trajectory data is gone. Phase B has nothing to compare.

Fix: Change grace period from 10 to 30 frames. Or switch from the custom SORT-like tracker to **ByteTrack** which uses Kalman filter prediction to maintain vehicle positions during occlusion. ByteTrack is built into YOLOv8:

```
# Replace Kishore's custom tracker call with:
results = model.track(
    frame,
    persist=True,
```

```
    tracker="bytetrack.yaml",  # better than custom SORT
    conf=0.30,
    verbose=False
)
```

ByteTrack was specifically designed to handle occlusions in crowded scenes — exactly your use case.

Problem 3 — Phase A Threshold is Fixed and Too Aggressive

What Kishore has: Hard threshold at **120 pixels** regardless of camera height, scene, or traffic density.

Two problems with this:

First, 120px is too small for a camera mounted high above a junction. At that angle, two vehicles in adjacent lanes that are 3 metres apart in real life might only be 80 pixels apart in the image. The threshold would flag them as collision candidates constantly even when they are safely separated.

Second, the threshold has no understanding of relative motion. Two vehicles 100px apart but moving away from each other are not a collision risk. Two vehicles 150px apart but converging rapidly are a serious risk. Distance alone is not enough.

Fix — add Time-To-Collision (TTC) as the real Phase A signal:

```
def time_to_collision(va, vb) -> float:
    """
    How many frames until these two vehicles meet
    if they continue at current velocity?
    Lower TTC = higher collision risk.
    """
    dx = va.centroid[0] - vb.centroid[0]
    dy = va.centroid[1] - vb.centroid[1]
    distance = np.sqrt(dx**2 + dy**2)

    # Relative velocity (closing speed)
```

```
    rel_vx = va.velocity[0] - vb.velocity[0]
    rel_vy = va.velocity[1] - vb.velocity[1]
    closing_speed = np.sqrt(rel_vx**2 + rel_vy**2)

    if closing_speed < 0.5:
        return float('inf')  # not converging

    return distance / closing_speed  # frames until contact

# Phase A gate: only flag pairs that are converging AND close
if distance < 150 and time_to_collision(va, vb) < 8:
    # These two vehicles will meet in under 8 frames    candidate
```

TTC = 8 frames at 10 FPS means they will collide in under 0.8 seconds. This is a real risk. Two vehicles 100px apart but moving parallel have TTC = infinity — correctly ignored.

Problem 4 — Velocity Shift Window is Too Small

What Kishore has: Compares last 3 frames vs frames -6 to -3. Total window = 6 frames = 0.6 seconds at 10 FPS.

Why this fails: Normal braking at a junction takes 0.5 to 1 second. Emergency crash braking takes 0.1 to 0.3 seconds. The 6-frame window cannot distinguish between these two cases because both look like a speed drop within that window.

Fix — use a longer baseline and a sharper drop criterion:

```
def is_emergency_stop(vehicle, baseline_frames=15, recent_frames=3):
    """
    Compare speed over last 3 frames vs speed over frames 5-20 ago.
    Emergency stop = drop of 80%+ in less than 3 frames
    Normal braking = gradual drop over 10+ frames
    """
    if len(vehicle.speed_history) < baseline_frames + recent_frames:
        return False

    # Speed well before the event
```

```
    baseline_speed = np.mean(
        vehicle.speed_history[-(baseline_frames + recent_frames):-recent_frames]
    )
    # Speed right now
    recent_speed = np.mean(vehicle.speed_history[-recent_frames:])

    if baseline_speed < 2.0:
        return False  # was already slow — ignore

    drop_percent = (baseline_speed - recent_speed) / baseline_speed * 100

    # Also check how fast the drop happened
    # (speed 5 frames ago vs now)
    mid_speed = np.mean(vehicle.speed_history[-8:-5])
    sudden = (mid_speed - recent_speed) / max(mid_speed, 0.1) > 0.65

    return drop_percent > 75 and sudden
```

This differentiates an emergency stop (sudden, from high speed) from normal deceleration (gradual, or from low speed).

Problem 5 — No Relative Velocity Between Vehicle Pairs

What Kishore has: Checks each vehicle's absolute speed drop individually.

What is missing: The speed drop of one vehicle means nothing without knowing what the other vehicle was doing. If both vehicles slow down together, that is normal traffic stopping. If Vehicle A was fast and Vehicle B was stationary and they suddenly have the same centroid position — that is a rear-end collision.

Fix — add relative velocity check to Phase B:

```
def relative_velocity_anomaly(va, vb):
    """
    If two vehicles that were moving at different speeds
    suddenly converge to the same speed and position = crash.
    """
```

```
    # Speed difference before (last 10-15 frames)
    prev_speed_diff = abs(
        np.mean(va.speed_history[-15:-5]) -
        np.mean(vb.speed_history[-15:-5])
    )

    # Speed difference now
    curr_speed_diff = abs(
        np.mean(va.speed_history[-3:]) -
        np.mean(vb.speed_history[-3:])
    )

    # If speeds were different before but now converged = they hit each other
    speeds_converged = (prev_speed_diff > 8.0) and (curr_speed_diff < 2.0)

    return speeds_converged
```

Problem 6 — Score Fusion Weights Are Uniform

What Kishore has: All signals feed into fusion but the weights are not published in tech_Des.md.

The problem with equal weights: Optical flow is a very noisy signal — a truck passing fast creates a spike even without any accident. If it has the same weight as trajectory stop (which is very precise), the noisy signal drowns out the precise one.

Research-backed weights from papers:

Signal	Weight	Reason
Trajectory stop after intersection	0.30	Most precise — from IITH 2018
TTC < threshold	0.20	Physics-based, low noise
Emergency speed drop	0.20	Good signal when baseline is long
CNN-LSTM confidence	0.15	Good but needs trained data
Optical flow spike	0.08	Noisy — weight lower
Flow angular dispersion	0.07	Good supplementary signal

Fix — update fusion with explicit weights:

```
WEIGHTS = {
    'trajectory_stop'   : 0.30,
    'ttc_critical'      : 0.20,
    'emergency_stop'    : 0.20,
    'cnn_lstm'          : 0.15,
    'optical_flow_spike': 0.08,
    'flow_dispersion'   : 0.07,
}

def fuse(scores: dict) -> float:
    return sum(WEIGHTS[k] * scores.get(k, 0.0) for k in WEIGHTS)
```

Problem 7 — Person Class Excluded

What Kishore has: Filters out pedestrians from detection to reduce overhead.

Why this is wrong for Coimbatore: A significant portion of Indian road accidents involve pedestrians being struck by vehicles — especially at junctions without signals. If person class is excluded, vehicle-pedestrian accidents are completely invisible to the system.

Fix: Add person back to the detection classes and add a specific vehicle-pedestrian proximity check in Phase A:

```
# In Phase A, treat vehicle-person pairs differently:
# Use smaller proximity threshold for vehicle-person pairs
# because a vehicle striking a person produces less overlap
# but is critically important to detect

if va.class_name in ['car', 'bike', 'auto', 'truck', 'bus'] and \
   vb.class_name == 'person':
    proximity_threshold = 80  # tighter for pedestrian check
else:
    proximity_threshold = 150  # standard vehicle-vehicle
```

Problem 8 — CNN-LSTM Has No Training Data

What Kishore has: ResNet18 + BiLSTM architecture defined in `model.py`. 126 lines.

The problem: The architecture exists but there is no mention of what data it was trained on. If it was not trained on accident videos, it outputs random probabilities. Untrained or randomly-initialised LSTM contributing 15-20% to the fusion score adds pure noise.

Fix: Either train it on the IITH dataset before including it in fusion, or temporarily set its weight to 0 in fusion until training is complete. A random LSTM score actively hurts accuracy.

```
# Until CNN-LSTM is properly trained:
WEIGHTS['cnn_lstm'] = 0.0
# Redistribute its weight to trajectory_stop
WEIGHTS['trajectory_stop'] = 0.45
```

Problem 9 — No Cooldown After Detection

What Kishore has: No mention of cooldown in tech_Des.md.

What happens without it: Once an accident is detected, every subsequent frame of the same accident — which still shows close vehicles, high flow, etc. — fires the alert again. One accident generates hundreds of Firebase uploads. The control room dashboard gets spammed.

Fix: After confirmed detection, suppress that camera for 20 seconds:

```
last_alert_time = {}

def should_alert(camera_id):
    now = time.time()
    if now - last_alert_time.get(camera_id, 0) < 20:
        return False
    last_alert_time[camera_id] = now
    return True
```

Summary Table — What to Fix, in Priority Order

Priority	Problem	Root Cause	Fix
1	No trajectory stop check	Intersection alone ≠ collision	Add post-intersection stop detection from IITH paper
2	Track grace period 10 frames	IDs lost during brief occlusion	Change to 30 frames or switch to ByteTrack
3	CNN-LSTM not trained	Random scores adding noise	Set weight to 0 until trained on IITH data
4	No cooldown	Alert spam per accident	Add 20-second camera cooldown
5	Fixed proximity threshold	Fires on dense normal traffic	Replace with TTC (Time-To-Collision)
6	Velocity window too short	Cannot separate braking from crashing	Extend baseline to 15 frames
7	No relative velocity check	Misses rear-end collisions	Add converging speed differential check
8	Equal fusion weights	Noisy signals dilute precise ones	Use research-backed weighted fusion
9	Person class excluded	Misses pedestrian accidents	Add person with tighter threshold

Fix problems 1, 2, 3, and 4 first. Those four alone will cut false positives by roughly 70%. Then tackle 5, 6, 7 using your data_logger threshold analysis to find the exact numbers for Coimbatore cameras specifically.